



Relacionamentos entre Classes

O que são relacionamentos entre classes?

- Em Programação Orientada a Objetos (POO), classes são abstrações de entidades do mundo real, como `Pessoa`, `Carro`, `ContaBancaria` e `Produto`.
- Assim como essas entidades se relacionam no cotidiano, classes também podem se vincular entre si.
- Esses vínculos são chamados **de relacionamentos entre classes** e indicam como uma classe **depende**, **utiliza** ou **está conectada** a outra.
- Relacionamentos tornam os sistemas mais **realistas**, **modulares** e **organizados**, favorecendo o reuso e a manutenção do código.

Exemplos do mundo real

- Uma Pessoa pode ter um Endereço.
- Um Carro possui um Motor.
- Uma Turma é composta por vários Alunos.
- Uma Empresa tem diversos Funcionários.
- Um Pedido envolve múltiplos Produtos.

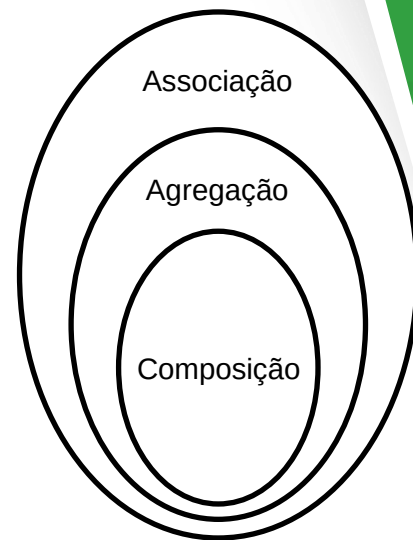
- Cada um desses exemplos pode representar um tipo diferente de relacionamento
 - Desde uma simples **associação** até uma **composição**, dependendo do nível de vínculo entre os objetos.

Para que servem os relacionamentos?

- Os relacionamentos entre classes têm como objetivos principais:
 - **Permitir a comunicação entre objetos** de diferentes classes.
 - **Refletir estruturas do mundo real** dentro do sistema.
 - **Reaproveitar código** ao conectar funcionalidades de diferentes classes.
 - **Organizar o sistema em partes coesas e colaborativas**, facilitando manutenção e entendimento.

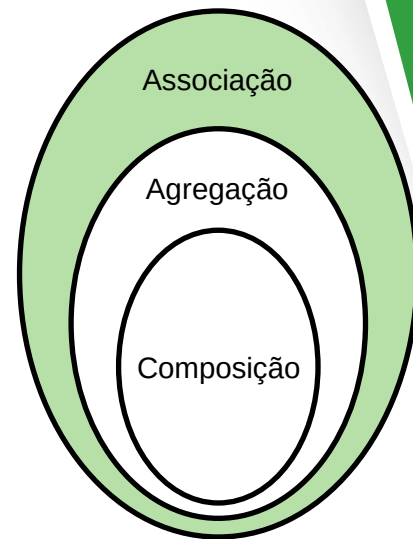
Tipos de Relacionamento

- Os três principais tipos de relacionamentos estruturais são:
 - **Associação:** Representa uma ligação entre duas classes, onde uma utiliza ou referencia a outra.
 - **Agregação:** Representa uma associação mais forte, em que uma classe possui outra, mas as instâncias podem viver separadamente.
 - **Composição:** Representa uma relação de dependência forte, onde uma classe "possui" outra de forma exclusiva, e seu ciclo de vida depende da classe principal.



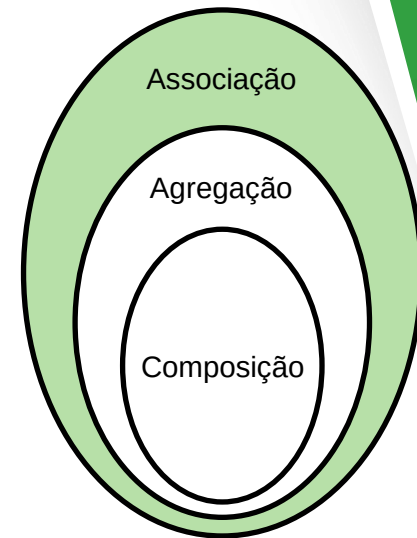
Associação

- Tipo mais simples de relacionamento entre classes na POO.
- Indica que uma instância de uma classe **conhece** ou **faz referência** a uma instância de outra.
- Não envolve vínculo de propriedade forte ou dependência de ciclo de vida.
- Representa uma **referência comum**: um objeto pode **interagir** com outro sem necessariamente "possuí-lo".



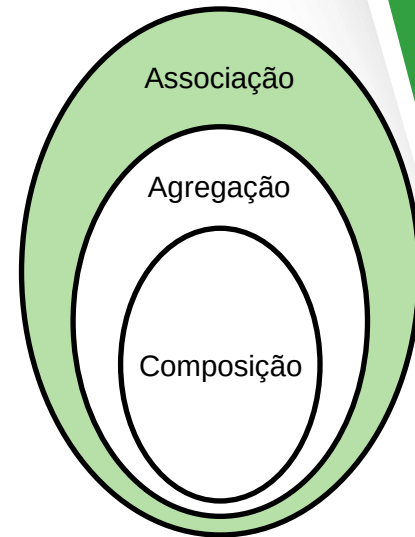
Associação – características

- A **associação** não implica dependência forte entre os objetos.
- Ela indica apenas que **um objeto possui uma referência a outro**.
- Em UML, é representada por **uma linha sólida simples conectando as classes**.



Associação – exemplo

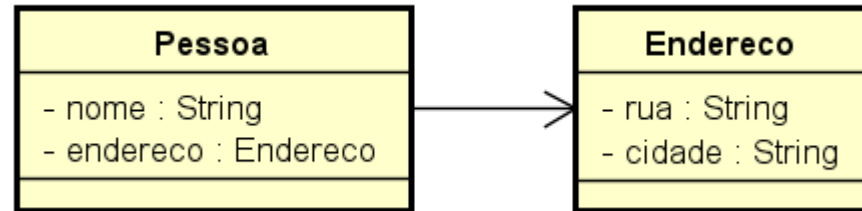
- Exemplo: uma **Pessoa** possui um **Endereço**.
- O **Endereço** pode ser compartilhado por várias pessoas (como membros da mesma família).
- Isso caracteriza uma associação entre as classes `Pessoa` e `Endereco`.
- Não há vínculo de propriedade exclusiva: ambas as classes existem de forma independente.



Associação – exemplo

```
public class Pessoa {  
    private String nome;  
    private Endereco endereco; // Associação com a classe Endereco  
}
```

```
public class Endereco {  
    private String rua;  
    private String cidade;  
}
```

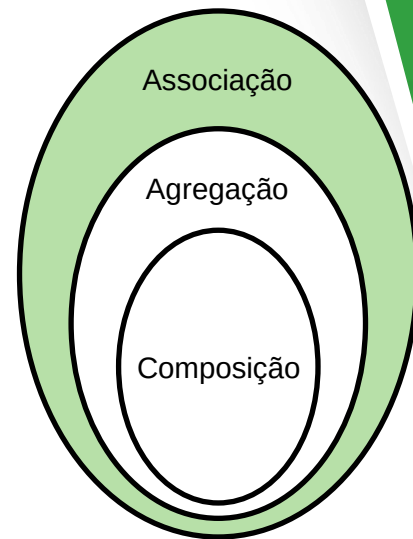


NOTA:

A seta indica a direção da associação, mostrando que a classe `Pessoa` conhece (ou faz referência a) a classe `Endereco`.

Associação Recursiva (Auto Referenciamento)

- **Associação Recursiva** ocorre quando **uma classe se relaciona com ela mesma**.
- Também chamada de **auto referenciamento**.
- É útil em **estruturas hierárquicas**, como:
 - Funcionários que têm um supervisor (que também é um funcionário).
 - Pastas que contêm subpastas.
- Uma instância da classe pode conter ou referenciar outra instância da mesma classe.

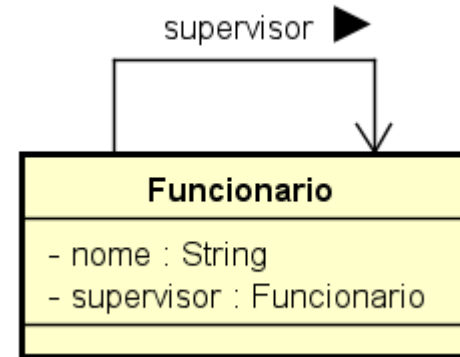


Associação Recursiva – exemplo

```
public class Funcionario {  
    private String nome;  
    private Funcionario supervisor; // Auto referenciamento  
}
```

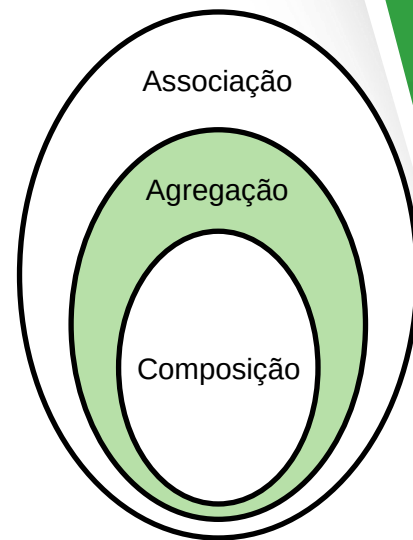
NOTA:

- A classe `Funcionario` possui um atributo `supervisor` do tipo `Funcionario`, o que indica que um funcionário pode ser subordinado a outro funcionário.
- Essa associação é útil em **estruturas hierárquicas**, como organogramas.



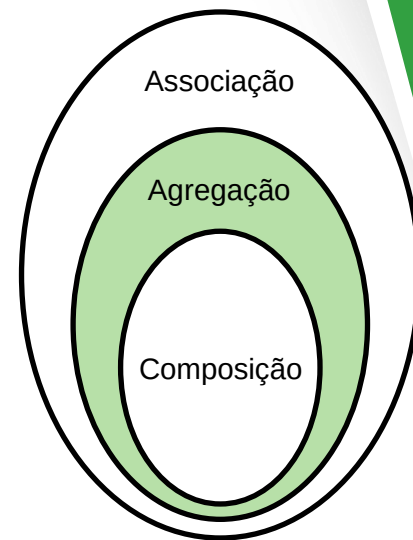
Agregação

- Tipo especial de associação com sentido de "**tem-um**", mas com **vida independente**.
- Indica que **uma classe agrupa ou contém outras**, sem ser dona delas.
- Os objetos agregados **podem existir separadamente** da classe que os agrega.
- Representa um **vínculo fraco de propriedade**, comum em situações como:
 - Um **Departamento** possui **Professores**, mas estes podem existir fora do **Departamento**.



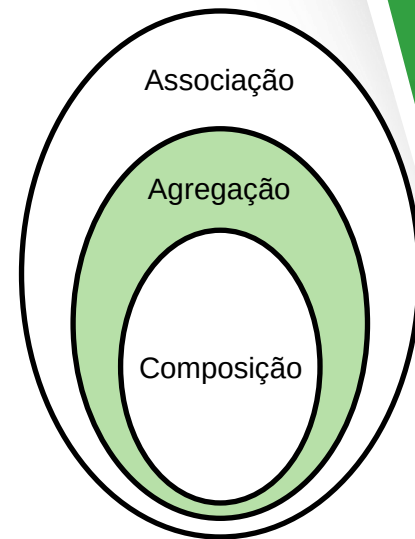
Agregação – características

- A **agregação** representa uma relação do tipo "**tem, mas não é dono**".
- O ciclo de vida dos objetos agregados é **independente** do ciclo de vida da classe agregadora.
- Em UML, esse relacionamento é representado por uma **linha com um losango não preenchido** na extremidade da classe agregadora.



Agregação – exemplo

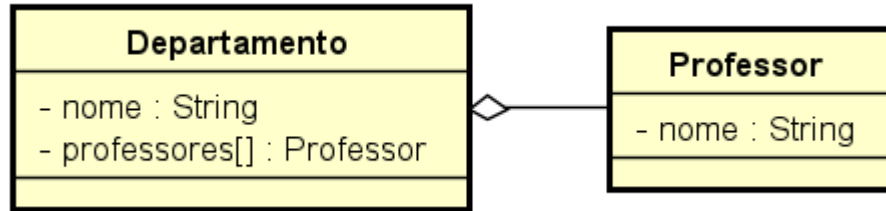
- Considere um **Departamento** de uma universidade.
- Ele pode agrupar vários **Professores**.
- No entanto, os **Professores não dependem do Departamento** para existir:
 - Podem ser transferidos para outro Departamento.
 - Continuam na universidade mesmo se o Departamento for extinto.
 - Podem atuar em mais de um Departamento.
- Essa independência caracteriza a **agregação** como uma relação de **propriedade fraca**.



Agregação – exemplo

```
public class Departamento {  
    private String nome;  
    private Professor[] professores; // Agregação  
}
```

```
public class Professor {  
    private String nome;  
}
```

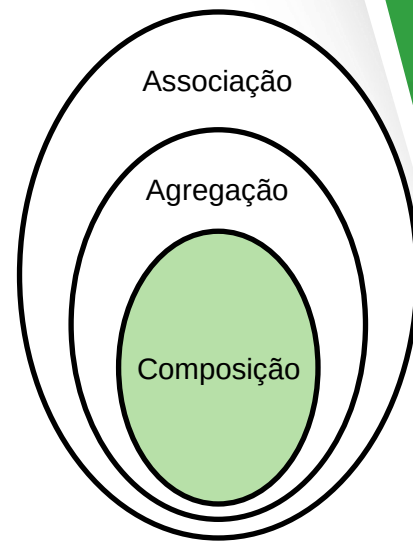


NOTA:

- A classe `Departamento` possui um array de `Professor`.
- O departamento "agrega" professores, mas os professores podem existir fora do departamento.
- O **losango não preenchido** no diagrama indica uma **relação fraca de propriedade**.

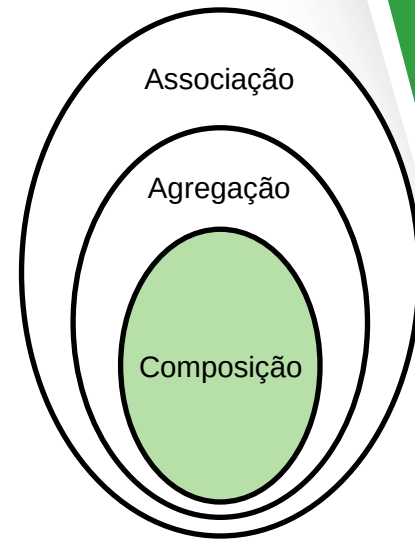
Composição

- É o tipo mais forte de relacionamento entre classes na Programação Orientada a Objetos.
- Indica uma relação em que **uma classe é composta por outras**, que não fazem sentido sozinhas.
- Os objetos "partes" **só existem enquanto o objeto principal existir**.
- Há **dependência total de ciclo de vida**: ao destruir o objeto principal, os objetos internos também são destruídos.
- Representa uma **relação de propriedade exclusiva e forte**.



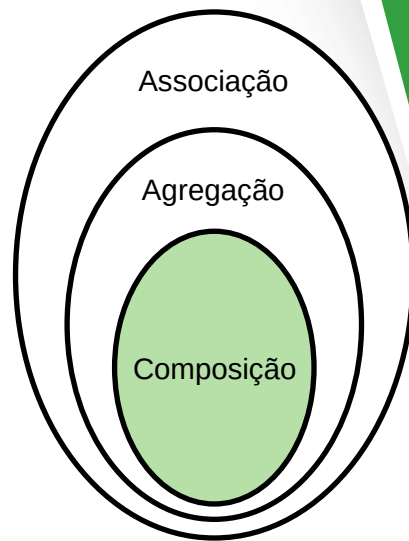
Composição – características

- A **composição** representa uma relação de "tem e é dono".
- O ciclo de vida das partes **depende completamente** do ciclo de vida do todo.
- Representa uma **relação de contenção forte**.
- Em UML, é representada por uma **linha com losango preenchido**.



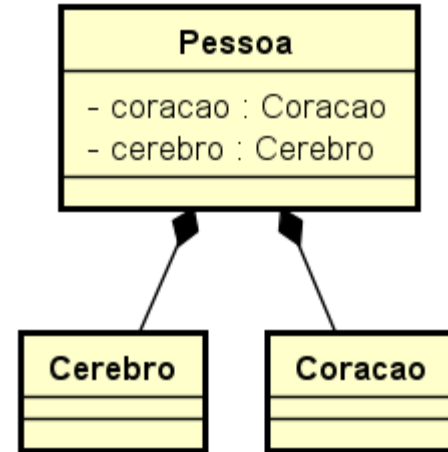
Composição – exemplo

- A classe `Pessoa` é composta por objetos do tipo `Coracao` e `Cerebro`.
- Esses órgãos **não fazem sentido isoladamente no sistema**
 - Existem apenas como parte de uma pessoa.
- Quando o objeto `Pessoa` é destruído, **seus órgãos também deixam de existir**.
- É uma relação de **dependência total e exclusiva**.



Composição – exemplo

```
public class Pessoa {  
    private Coracao coracao;  
    private Cerebro cerebro;  
}  
  
class Coracao {  
    // atributos e métodos do coração  
}  
  
class Cerebro {  
    // atributos e métodos do cérebro  
}
```



NOTA:

- Pessoa é composta por Coracao e Cerebro.
- Esses componentes são criados junto com o objeto Pessoa e não existem fora dele.
- Se a Pessoa for destruída, **seus componentes também serão**.
- O **losango preenchido** no diagrama representa um **vínculo de dependência total**.

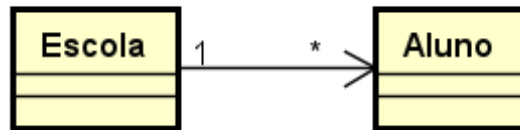
Multiplicidade (ou Cardinalidade)

- Multiplicidade define **quantas instâncias de uma classe** podem estar associadas a outra.
- Também conhecida como **cardinalidade** na modelagem UML.
- Indica os **limites mínimo e máximo** permitidos ou exigidos no relacionamento.
- Exemplo:
 - Uma Pessoa pode ter **apenas um CPF**.
 - Uma Escola pode ter **vários Alunos**.
- Essencial para modelar restrições e relações **realistas e precisas** entre objetos.

Multiplicidade (ou Cardinalidade)

- Exemplos comuns de notação de multiplicidade:

- 1: exatamente uma instância.
- 0..1: zero ou uma instância (opcional).
- 0..* ou *: muitas instâncias (zero ou mais).
- 1..*: uma ou mais instâncias.
- 3..5: entre três e cinco instâncias (restrição específica).



- Multiplicidade nos três tipos de relacionamentos:

- Associação:** Uma Pessoa pode estar associada a **1** ou **0..1** Endereco.
- Agregação:** Um Departamento pode agregar ***** Professores.
- Composição:** Uma Pessoa possui **1** Coracao e **1** Cerebro.

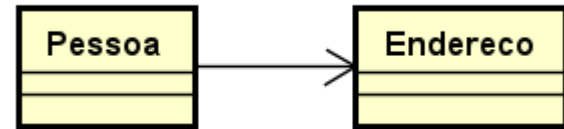
Navegabilidade

- Define a **direção da comunicação** entre objetos de classes relacionadas.
- Indica **qual classe possui uma referência** para acessar a outra.
- A classe com a referência pode "**ver**" e **interagir** com a outra.
- Representada em UML por uma **seta de ponta aberta** (→).
- A seta aponta da classe que **conhece** para a classe **referenciada**.
- Exemplo:
 - Pedido → Cliente: um pedido conhece seu cliente, mas o cliente não precisa conhecer todos os pedidos.

Navegabilidade

- Tipos de navegabilidade:

- **Unidirecional:** apenas uma classe conhece a outra.
- **Bidirecional:** ambas as classes conhecem uma à outra (sem seta ou com setas nos dois sentidos).

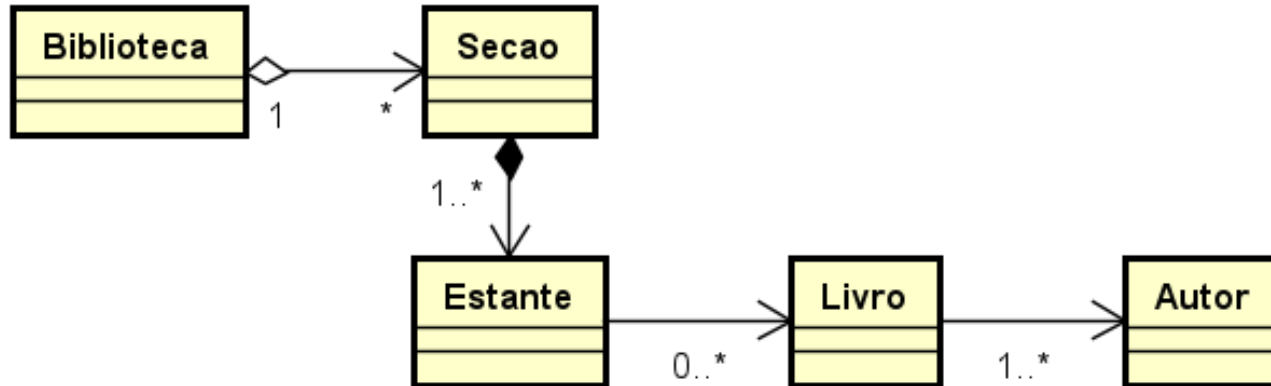


- Exemplos práticos de navegabilidade:

- Pessoa → Endereco: a classe Pessoa possui uma referência a Endereco. Endereco não sabe nada sobre Pessoa.
- Pedido → Cliente: um pedido conhece o cliente associado, mas o cliente não precisa conhecer diretamente todos os pedidos.
- Departamento → Professor: o departamento pode acessar os professores, mas os professores não precisam acessar o departamento.

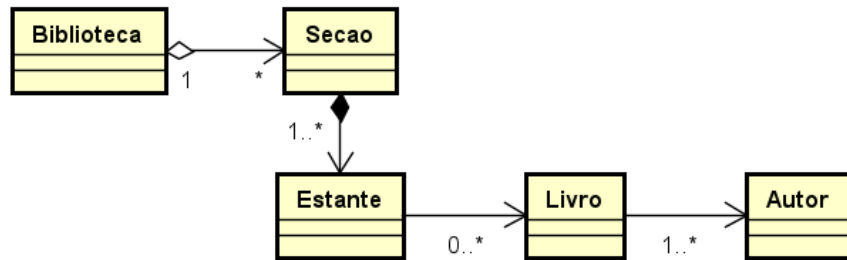
Exemplo completo:

Sistema de Biblioteca com Seções, Estantes e Livros



Exemplo completo:

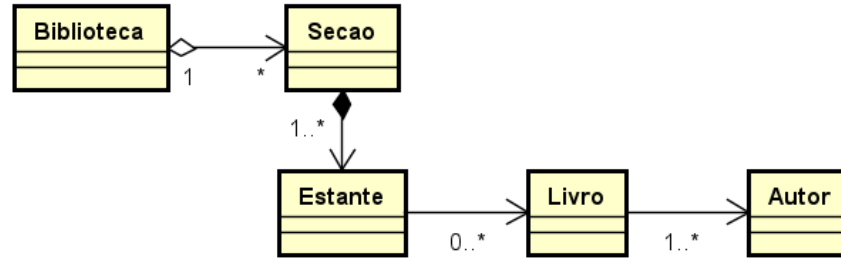
Sistema de Biblioteca com Seções, Estantes e Livros



- **Agregação:** A classe `Biblioteca` agrega múltiplas instâncias da classe `Secao`. As seções podem existir independentemente da biblioteca.
- **Composição:** Cada `Secao` é composta por uma ou mais `Estante`. As estantes não fazem sentido fora da seção a que pertencem.
- **Associação:**
 - `Estante` está associada a vários `Livro`.
 - `Livro` está associado a um `Autor`, mas o autor não conhece os livros diretamente (navegabilidade unidirecional).

Exemplo completo:

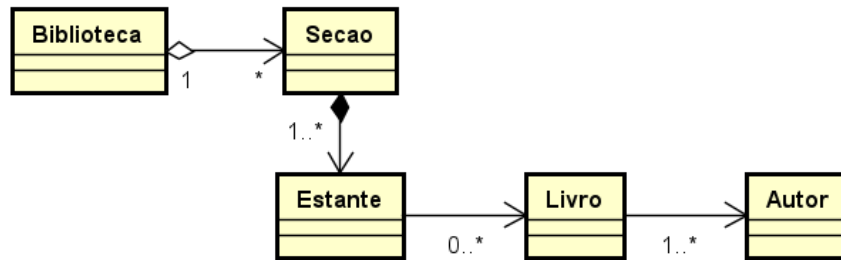
Sistema de Biblioteca com Seções, Estantes e Livros



- **Multiplicidade:**
 - Biblioteca → 1..* Secao
 - Secao → 1..* Estante
 - Estante → 0..* Livro
 - Livro → 1..* Autor

Exemplo completo:

Sistema de Biblioteca com Seções, Estantes e Livros



- **Navegabilidade:**

- A navegação ocorre da Biblioteca para a Secao, da Secao para a Estante, da Estante para o Livro e do Livro para o Autor.
- Isso significa que cada classe possui referências explícitas às outras, mas não o contrário.

Exemplo completo:

Sistema de Biblioteca com Seções, Estantes e Livros

```
public class Biblioteca {  
    private String nome;  
    private Secao[] secoes; // Agregação  
}
```

```
public class Secao {  
    private String nome;  
    private Estante[] estantes; // Composição  
}
```

```
public class Estante {  
    private int numero;  
    private Livro[] livros; // Associação  
}
```

```
public class Livro {  
    private String titulo;  
    private Autor[] autores; // Associação  
}
```

```
public class Autor {  
    private String nome;  
    private String nacionalidade;  
}
```

Exercícios propostos

Verificar atividade no Moodle